

Incident Report: PowerShell AMSI Detection & Automated Prevention

Scenario: Suspicious PowerShell Script Execution
Platform: Microsoft Defender for Endpoint
Detection Method: Antimalware Scan Interface (AMSI)
Status: Resolved
Severity: Low
Test Type: Controlled Security Simulation

1. Executive Summary

A controlled Microsoft AMSI demonstration was performed on a Windows 11 endpoint onboarded to Microsoft Defender for Endpoint. A PowerShell script containing a Microsoft test detection pattern was executed. During execution, the script content was inspected through the Antimalware Scan Interface (AMSI), where Microsoft Defender detected the signature:

Virus:Win32/MpTest!amsi

Microsoft Defender automatically prevented the script from executing. The detection was subsequently reported to Microsoft Defender for Endpoint, where an alert and incident were generated. The incident was investigated by analyzing the alert details, affected endpoint, user, PowerShell command line, and alert timeline. The activity was confirmed as an authorized security test, classified as a benign positive, and resolved.

2. Lab Environment

Component	Configuration
Endpoint	Windows 11 Virtual Machine
Endpoint Protection	Microsoft Defender Antivirus
EDR Platform	Microsoft Defender for Endpoint
Detection Mechanism	AMSI
Scripting Engine	PowerShell
Device	WIN11
User	WIN11\vboxuser

3. Objective

The objective of this scenario was to validate Microsoft Defender for Endpoint's ability to:

- Monitor PowerShell activity.
 - Inspect script content using AMSI.
 - Detect suspicious or malicious script content.
 - Automatically prevent execution.
 - Generate endpoint alerts.
 - Create security incidents.
 - Support SOC investigation and incident resolution.
-

4. Attack Simulation

A controlled Microsoft AMSI demonstration was executed using PowerShell.

The script involved in the simulation was:

AMSI_PoSh_script.ps1

The PowerShell execution included:

-ExecutionPolicy Bypass

The simulation followed the workflow below:

PowerShell Script Execution -> AMSI Content Inspection -> Microsoft Defender Antivirus -> MpTest Signature Detected -> Execution Prevented -> Defender for Endpoint Alert -> Incident Created

The simulation did not involve real malware. The MpTest signature was intentionally triggered as part of a controlled Microsoft security demonstration.

5. Detection & Telemetry

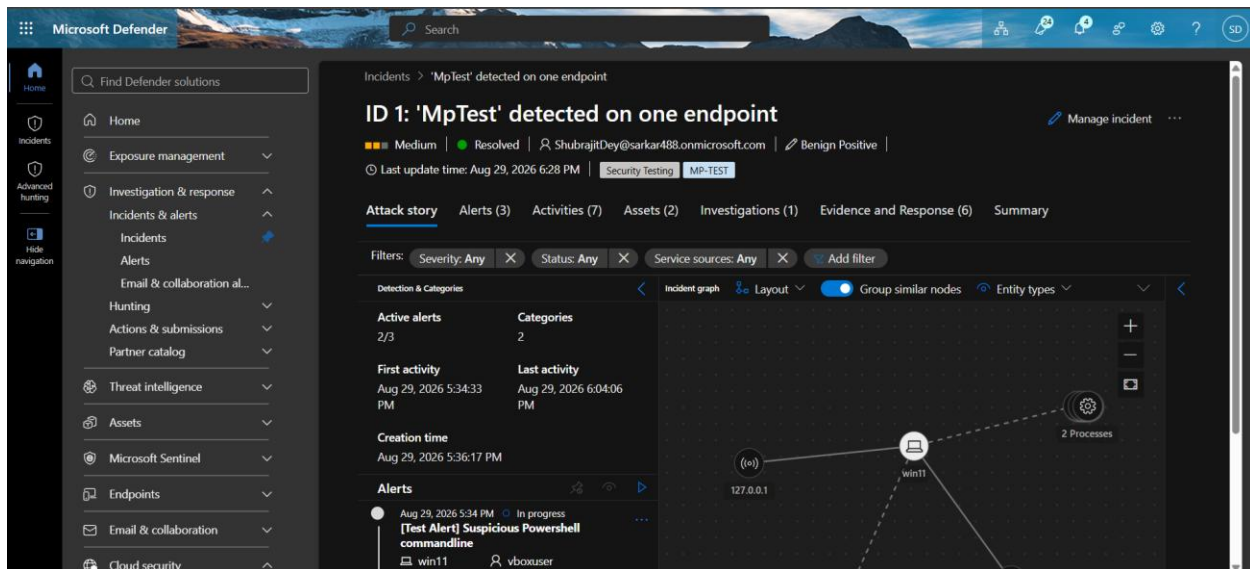
Microsoft Defender detected the following signature:

Virus:Win32/MpTest!amsi

The alert indicated that an active MpTest detection was prevented from executing through AMSI.

The investigation identified:

Investigation Item	Finding
Initiating Process	powershell.exe
Script	AMSI_PoSh_script.ps1
Execution Parameter	-ExecutionPolicy Bypass
Detection	Virus:Win32/MpTest!amsi
Defender Action	Execution Prevented
Remediation Result	Success



6. Incident Timeline

The following sequence of events was observed during the simulation:

PowerShell Process Started -> AMSI_PoSh_script.ps1 Executed -> Script Content Submitted to AMSI -> Microsoft Defender Analyzed Content -> Virus:Win32/MpTest!amsi Detected -> Execution Automatically Prevented -> Alert Generated -> Incident Created -> SOC Investigation Performed -> Incident Classified and Resolved

7. Investigation Findings

The incident was investigated using Microsoft Defender for Endpoint.

The following evidence was identified:

Affected Device

WIN11

Affected User

WIN11\vboxuser

Initiating Process

powershell.exe

Suspicious Command-Line Parameter

-ExecutionPolicy Bypass

Executed Script

AMSI_PoSh_script.ps1

Threat Detection

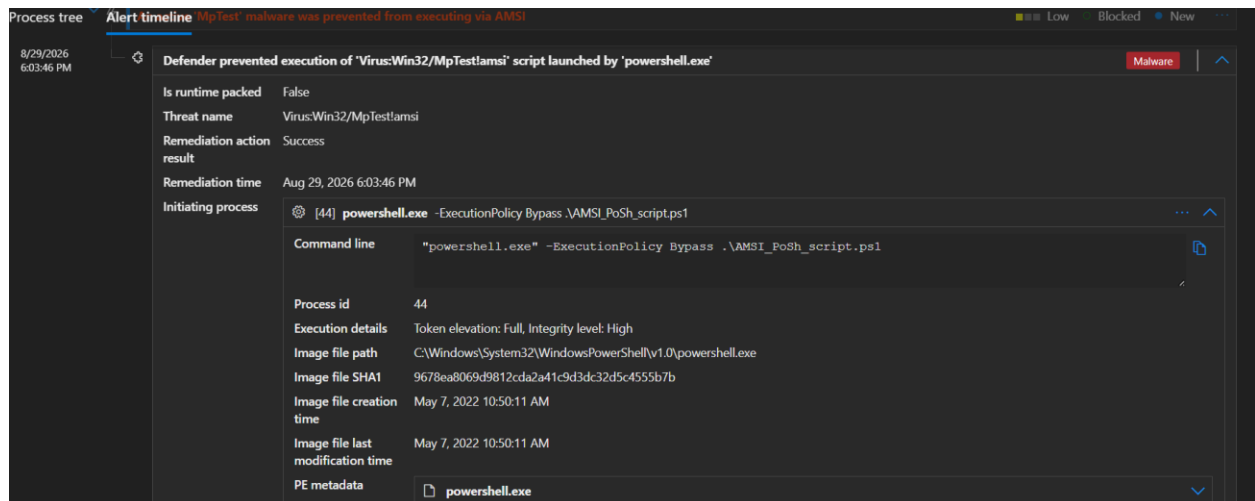
Virus:Win32/MpTest!amsi

Response

Microsoft Defender prevented execution of the detected script.

Remediation Status

Success



8. Detection Analysis

The primary detection mechanism used in this scenario was AMSI.

AMSI enables applications such as PowerShell to submit script content for inspection by antimalware solutions.

The detection chain was:

powershell.exe



AMSI



Microsoft Defender Antivirus



MpTest Detection



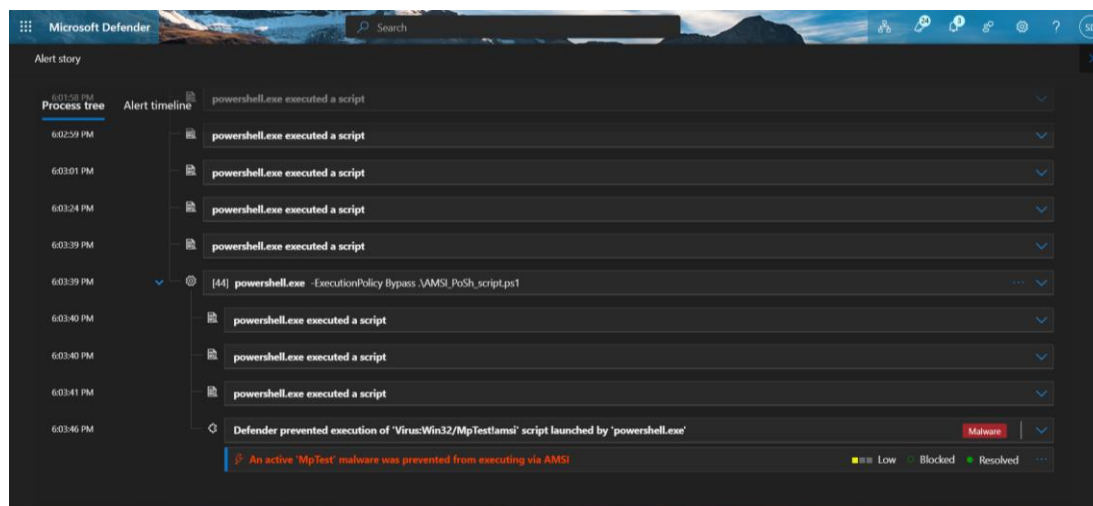
Execution Blocked

The investigation also identified the use of:

-ExecutionPolicy Bypass

This parameter can be suspicious because it may allow PowerShell scripts to run without being restricted by the configured execution policy.

However, the use of this parameter alone does not confirm malicious activity. In this scenario, it became relevant because the script content was subsequently detected by Microsoft Defender.



9. Automatic Prevention and Remediation

The malicious test pattern was automatically prevented from executing by Microsoft Defender.

The response occurred directly on the endpoint:

Suspicious Script



AMSI Inspection



Microsoft Defender Detection



Execution Automatically Blocked



Remediation Successful

No manual containment action was required.

The prevention mechanism was performed by Microsoft Defender's endpoint protection capabilities and was not triggered through a Microsoft Sentinel playbook.

Microsoft Defender for Endpoint then provided the security operations visibility by generating the alert and incident.

10. MITRE ATT&CK Mapping

Tactic	Technique	Observed Activity
Execution	Command and Scripting Interpreter	PowerShell script execution
Execution	PowerShell	powershell.exe initiated the script
Defense Evasion	Execution Policy Bypass	-ExecutionPolicy Bypass observed

Note: This mapping represents the observed behavior during the controlled simulation and does not indicate an actual compromise.

11. Impact Assessment

No actual endpoint compromise occurred.

The detected activity was part of an authorized Microsoft security demonstration.

Microsoft Defender successfully prevented execution before the simulated activity could continue.

No evidence of the following was identified during the investigation:

- Successful malware execution
 - Persistent malicious activity
 - Unauthorized payload execution
 - Endpoint compromise
 - Manual containment requirement
-

12. Response Actions

The following SOC investigation steps were performed:

1. Reviewed the generated incident.
 2. Analyzed the associated alert.
 3. Identified the affected endpoint.
 4. Identified the affected user.
 5. Investigated the initiating PowerShell process.
 6. Reviewed the PowerShell command-line activity.
 7. Analyzed the alert timeline.
 8. Confirmed AMSI detection.
 9. Verified automatic execution prevention.
 10. Confirmed successful remediation.
 11. Classified the activity as authorized testing.
 12. Resolved the incident.
-

13. Incident Classification

The detection was confirmed as valid because Microsoft Defender successfully detected the intended MpTest signature.

However, the activity was intentionally generated as part of a controlled security test.

The incident was therefore classified as:

Benign Positive

The incident was subsequently marked as:

Resolved

14. Recommendations

Based on the findings from this scenario:

- Continue monitoring PowerShell activity on endpoints.
 - Investigate suspicious PowerShell command-line arguments.
 - Maintain AMSI integration and endpoint protection.
 - Verify remediation actions after automated prevention.
 - Use endpoint telemetry to investigate related process activity.
 - Clearly distinguish authorized security testing from genuine malicious activity.
 - Use Advanced Hunting for deeper investigation during higher-severity incidents.
-

15. Conclusion

This scenario successfully demonstrated endpoint detection, automatic prevention, and SOC investigation using Microsoft Defender for Endpoint.

A controlled PowerShell AMSI simulation triggered the Virus:Win32/MpTest!amsi detection. Microsoft Defender automatically prevented the script from executing and successfully remediated the detected activity.

Microsoft Defender for Endpoint generated an alert and incident, allowing the activity to be investigated through endpoint telemetry, command-line analysis, and timeline investigation.

The activity was confirmed as authorized security testing, classified as a benign positive, and resolved.
